CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Fundamentos de arquitectura de software

Arquitectura y Diseño de Software · Jesús Ariel González Bonilla · 2026-B · Corte 1 · Unidad 1

Patrones y estilos arquitectónicos

Un sistema puede funcionar perfectamente y estar mal construido

¿Cuántos sistemas conocen que funcionan… y que nadie se atreve a tocar?

- Cumple todos los requisitos funcionales
- Nadie quiere desplegar un viernes
- Un cambio pequeño toma tres semanas

Lo que vas a poder hacer al terminar la unidad

- Distinguir arquitectura de diseño detallado
- Justificar decisiones con atributos de calidad
- Aplicar modularidad, separación y abstracción
- Separar patrón arquitectónico de patrón GoF
- Elegir entre monolito y distribuido con criterio
- Explicar Hexagonal, Onion y CQRS, y escribir un ADR

01

1. Qué es, y qué no es, la arquitectura

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

La arquitectura es lo que cuesta caro cambiar

> **Si una decisión se revierte en una tarde, es diseño. Si revertirla cuesta tres meses y una migración de datos, es arquitectura.**

- **No la define el diagrama**
- **No la define el cargo de quien decide**
- **La define el costo de deshacerla**

Arquitectura y diseño no son dos nombres para lo mismo

Criterio	Arquitectura	Diseño detallado
Decide sobre	Componentes, límites, protocolos, despliegue	Clases, métodos, estructuras de datos
Impacto	Todo el sistema y sus equipos	Un módulo, a veces un archivo
Revertir cuesta	Semanas o meses	Horas o días
La guía	Atributos de calidad (RNF)	Requerimientos funcionales
Ejemplo	«Separamos reportería y la alimentamos por eventos»	«Extraemos el cálculo a `TarifaCalculator`»

Un atributo de calidad sin número no sirve para decidir

No es un requisito

- "El sistema debe ser rápido"
- No descarta ninguna arquitectura
- Nadie puede comprobarlo

Sí es un requisito

- "Ingreso y salida en menos de 2 s en el p95"
- Descarta arquitecturas completas
- Se mide y se incumple

Los siete atributos que vas a usar en el taller

Atributo	Pregunta que responde	Se mide con
Rendimiento	¿Cuánto tarda una operación crítica?	Latencia p95/p99
Escalabilidad	¿Qué pasa si se duplica la carga?	Curva carga/recursos
Disponibilidad	¿Cuánto puede estar caído?	% mensual, MTBF/MTTR
Mantenibilidad	¿Cuánto cuesta un cambio típico?	Tiempo de ciclo
Testabilidad	¿Puedo probar el negocio sin infraestructura?	Tests sin BD
Seguridad	¿Quién puede hacer qué y con qué datos?	Controles
Observabilidad	¿Puedo saber qué pasó sin adivinar?	Logs, métricas, trazas

PREGUNTA

Según el criterio visto, ¿qué convierte a una decisión en "arquitectónica"?

A Que aparezca dibujada en un diagrama UML del proyecto

B Que revertirla más adelante resulte costoso o disruptivo

C Que la tome el arquitecto y no el equipo de desarrollo

D Que involucre elegir un lenguaje de programación

02

2. Los tres principios fundamentales

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Modularidad: alta cohesión, bajo acoplamiento

Bien trazado

- Tarifas y Pagos separados
- Una interfaz, un contrato
- Cambiar tarifas NO toca pagos

Mal trazado

- Tarifas + Pagos + SMS + Reportes juntos
- Todo se llama a todo
- Cambiar algo rompe otra cosa

Separación de preocupaciones: política y mecanismo

Política

Las reglas del negocio. Cómo se calcula una tarifa nocturna

Mecanismo

Los detalles técnicos. Que se guarde en PostgreSQL y se avise por SMS

La prueba

¿Puede existir la política sin el mecanismo?

Si no puede

La separación falló, aunque haya carpetas separadas

Abstracción: exponer el qué, ocultar el cómo

ABSTRACCIÓN CORRECTA

```
// El núcleo declara QUÉ necesita, en su propio lenguaje.
public interface PasarelaDePagos {
    ResultadoPago cobrar(Dinero monto, MedioDePago medio);
}

public class CerrarSalidaDeVehiculo {
    private final PasarelaDePagos pasarela;   // depende de la abstracción
```

ABSTRACCIÓN CON FUGA

```
// La firma arrastra el vocabulario del proveedor: ya no abstrae.
public interface PasarelaDePagos {
    StripeChargeResponse charge(StripeChargeRequest req, String apiKey)
        throws StripeException;
}
// Cambiar de proveedor obliga a tocar TODO el negocio que la usa.
```

Una interfaz no es automáticamente una abstracción

> **Si la interfaz habla el idioma del proveedor —`StripeChargeRequest`, `apiKey`, `StripeException`— solo cambiaste el lugar del acoplamiento; no lo eliminaste.**

- Se llama *leaky abstraction*
- Se detecta leyendo la firma, no la implementación
- Es el error más frecuente al «desacoplar»

PREGUNTA

Un equipo define `PasarelaDePagos` con el método `charge(StripeChargeRequest req, String apiKey)`. ¿Qué problema tiene?

A Ninguno: al ser una interfaz, el núcleo ya queda desacoplado

B Es una abstracción con fuga: expone el vocabulario del proveedor

C Viola CQRS porque mezcla comandos con consultas

D Rompe la cohesión del módulo de pagos

03

3. Decidir es renunciar

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Netflix no prueba que microservicios sea mejor

Caso	Decisión	Efecto
Android	Construir sobre el kernel de Linux	Compatibilidad con hardware diverso
Netflix	Migrar de monolito a microservicios	Escalado y despliegue independientes
Amazon	Descomponer en servicios	Agilidad interna y, con el tiempo, AWS
Facebook	Pasar a componentes especializados	Sostuvo el crecimiento sin perder rendimiento

Prueban que esas organizaciones, con ese tamaño y ese tráfico, recibieron más beneficio que costo.

Toda decisión compra algo y lo paga con otra cosa

Decisión	Compra	Paga con
Separar en servicios	Escalado y despliegue independientes	Latencia, consistencia, operación
Caché de lecturas	Rendimiento	Datos potencialmente obsoletos
Mensajería asíncrona	Desacople y resiliencia	Depuración y orden
Base de datos única	Consistencia fuerte, consultas simples	Punto único de contención y falla
Monolito modular	Simplicidad operativa	Escalado grueso, todo o nada

Sin alternativas descartadas, no hubo decisión

1

Contexto

El hecho medible que obliga a decidir. Con número

2

Decisión

Qué se hace, en una frase

3

Alternativas

Qué se descartó y por qué. Es la sección que vale

4

Consecuencias

Lo que mejora Y lo que se complica

Así se ve un ADR terminado

```
ADR-007 — Separar reportería del core operacional
_____________________________________________________

Estado      : Aceptada (2026-03-14)
Contexto    : Los reportes bloquean la BD; el ingreso supera
              8 s en hora pico. El RNF2 exige < 2 s.
Decisión    : Reportería sobre réplica de lectura alimentada
              por eventos. El core publica, no consulta.
Alternativas: (a) Optimizar índices — insuficiente: el problema
                  es contención, no plan de consulta.
              (b) Réplica síncrona — no elimina el impacto
                  sobre las escrituras.
Consecuencias: + La operación crítica deja de degradarse.
               - Los reportes quedan consistentes en ~30 s.
               - Aparece un pipeline nuevo que hay que operar.
```

PREGUNTA

¿Qué sección de un ADR distingue una decisión real de una simple preferencia?

A El estado (propuesta, aceptada, reemplazada)

B La fecha en que se tomó y quién la firmó

C Las alternativas descartadas y la razón del descarte

D El diagrama que acompaña al documento

04

4. Patrones: dos niveles que no se deben mezclar

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Patrón arquitectónico y patrón GoF operan en niveles distintos

	Patrón arquitectónico	Patrón de diseño (GoF)
Organiza	Componentes y capas del sistema	Clases y objetos
Alcance	Todo el sistema	Un módulo
Decide	Dónde vive la lógica, quién llama a quién	Cómo se resuelve un problema local
Ejemplos	Capas, MVC, Microservicios, Hexagonal, CQRS	Singleton, Factory, Adapter, Observer
Cambiarlo cuesta	Alto	Bajo

Los tres patrones arquitectónicos del temario

Capas

Niveles jerárquicos; cada uno conoce solo al inferior. El más difundido

MVC

Modelo, vista y controlador. Nació para interfaces; hoy en servidor y en el front

Front-Back separado

Cliente y servidor se despliegan aparte, unidos por una API

El riesgo de Capas

Basta que un controlador consulte la BD para romperlo

Las tres familias de patrones GoF

Familia	Qué resuelve	Del temario
Creacionales	Cómo se crean los objetos	Singleton, Factory Method, Abstract Factory
Estructurales	Cómo se componen los objetos	Adapter, Decorator, Composite
De comportamiento	Cómo interactúan y reparten responsabilidad	Observer, Strategy, Command

Strategy: una familia de algoritmos intercambiables

TARIFAS POR MODALIDAD

```
public interface PoliticaDeTarifa {
    Dinero calcular(Duration permanencia, TipoVehiculo tipo);
}

public class TarifaNocturna implements PoliticaDeTarifa { /* ... */ }
public class TarifaMensual  implements PoliticaDeTarifa { /* ... */ }

public class Ticket {
    private final PoliticaDeTarifa politica;
    public Dinero calcularTotal() { return politica.calcular(permanencia(), tipo()); }
}
```

Adapter y Observer: integraciones y notificaciones

ADAPTER

```
// Traduce la interfaz del tercero al lenguaje del núcleo.
public class AdaptadorPasarelaX implements PasarelaDePagos {
    private final ClienteX clienteX;          // SDK del proveedor
    @Override public ResultadoPago cobrar(Dinero monto, MedioDePago medio) {
        var r = clienteX.charge(monto.enCentavos(), medio.token());
        return r.ok() ? ResultadoPago.exitoso(r.id()) : ResultadoPago.fallido(r.error());
    }
}
```

OBSERVER

```
// El core publica un hecho; quien quiera reaccionar se suscribe.
public class SalidaRegistrada implements EventoDeDominio {
    public final UUID ticketId;
    public final Instant momento;
}
// Suscriptores: NotificadorSms, ActualizadorCupos, ProyeccionReportes.
// El core no sabe cuántos escuchan ni qué hacen.
```

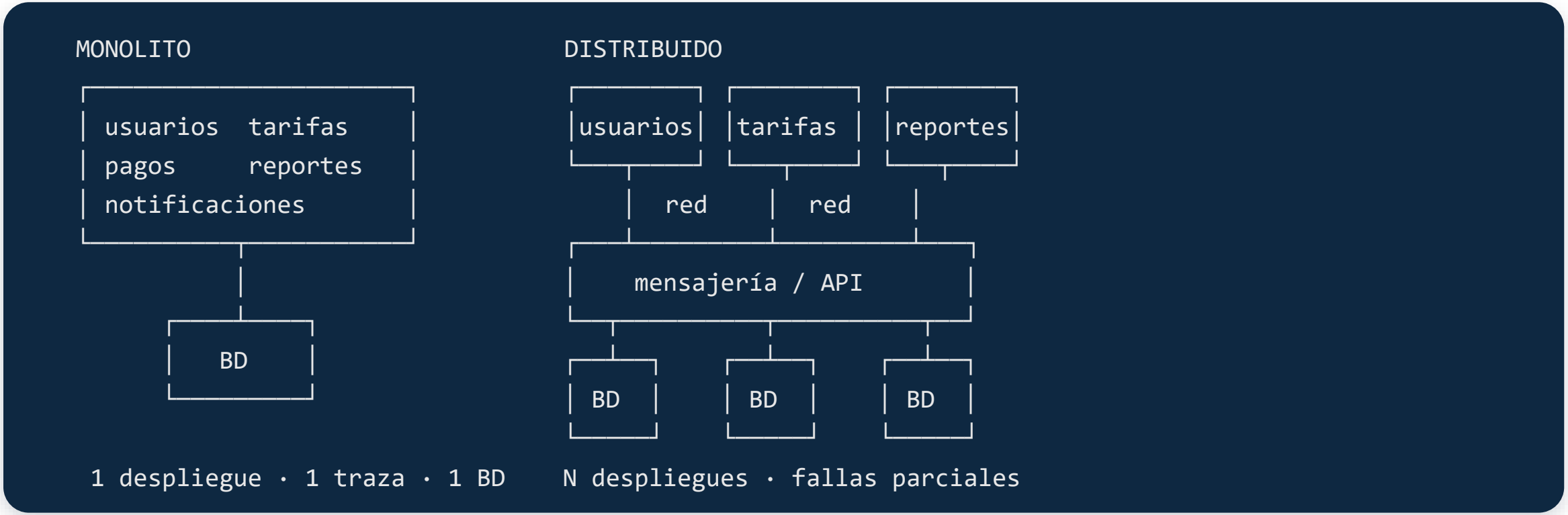
05

5. Monolito o distribuido

El prestigio no es un criterio de diseño

Criterio	Monolito	Distribuido
Despliegue	Una unidad: simple, todo o nada	Independiente, pero coordinado
Escalado	Se replica el sistema completo	Solo lo que lo necesita
Latencia interna	Memoria (ns)	Red (ms), y puede fallar
Consistencia	Transacción ACID natural	Eventual: Saga o compensación
Depuración	Una traza, un proceso	Trazado distribuido obligatorio
Costo operativo	Bajo	Alto
Equipo mínimo	Pequeño	Varios equipos con autonomía

Cómo se ven, uno al lado del otro



La tercera opción suele ser la respuesta

Monolito modular · Un solo despliegue, fronteras internas estrictas

Módulos con interfaz · Nadie consulta las tablas de otro módulo

Datos por módulo · Mismo motor, esquemas separados

Extracción después · Se saca lo que cambia mucho o lo que pesa mucho

El peor resultado posible: el monolito distribuido

Servicios separados que igual deben desplegarse juntos porque comparten base de datos y se llaman en cadena.

- Se paga la latencia de red
- Se paga la complejidad operativa
- Se paga la falla parcial
- No se recibe escalado ni despliegue independiente

PREGUNTA

ParkKbus debe responder en menos de 2 s, tiene 2 desarrolladores y no puede parar más de 2 horas. ¿Qué enfoque es el más defendible?

A Microservicios completos, porque es lo que hizo Netflix

B Mantener el monolito tal como está y solo optimizar índices

C Monolito modular y extraer después lo que más lo justifique

D Reescribir el sistema desde cero con otro lenguaje

PREGUNTA

Un equipo separa el sistema en seis servicios, pero todos comparten la base de datos y se despliegan juntos. ¿Qué obtuvo?

A Una arquitectura de microservicios correctamente implementada

B Un monolito modular con fronteras internas bien trazadas

C Un monolito distribuido: costos de lo distribuido sin sus beneficios

D Una implementación de CQRS con consistencia fuerte

06

6. Los estilos arquitectónicos modernos

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Los tres protegen exactamente lo mismo

Hexagonal

Puertos y adaptadores. El núcleo declara qué necesita

Onion

Anillos concéntricos. Hacia adentro, más abstracto

CQRS

Separa el modelo que escribe del que lee

En común

Proteger las reglas del negocio de los detalles técnicos

Hexagonal: el núcleo declara, los adaptadores implementan

REST / CLI / Job

Adaptadores primarios

Puertos de entrada

Lo que la app sabe hacer

Núcleo

Entidades, casos de uso, reglas

Puertos de salida

Lo que la app necesita

PostgreSQL / Pasarela

Adaptadores secundarios

Las flechas de dependencia siempre apuntan hacia adentro. El núcleo no importa nada de infraestructura.

Onion: la misma regla, dibujada como anillos

Infraestructura / UI / Persistencia · Concreto, reemplazable

Servicios de aplicación · Los casos de uso

Servicios de dominio · Reglas que cruzan entidades

Entidades · El corazón. Nada depende hacia afuera

CQRS: separar lo que escribe de lo que lee

Comandos

- Cambian el estado
- Modelo rico, con invariantes
- Se optimiza para consistencia y auditoría

Consultas

- No cambian nada
- Modelo plano, a la medida de la pantalla
- Se optimiza para latencia y volumen

No obliga a dos bases de datos ni a Event Sourcing: se puede aplicar separando solo los modelos.

Cuándo usar cada uno, y cuándo no

Estilo	Úsalo cuando	Evítalo cuando
Hexagonal	Hay muchas integraciones y quieres testear sin ellas	El sistema es un CRUD delgado sin reglas propias
Onion	El dominio es rico y hay invariantes que proteger	La lógica vive en la base de datos y así seguirá
CQRS	Las lecturas son masivas y golpean a las escrituras	Lecturas y escrituras tienen cargas parecidas

PREGUNTA

En arquitectura Hexagonal, ¿hacia dónde apuntan las dependencias?

A Del núcleo hacia los adaptadores de infraestructura

B De los adaptadores hacia el núcleo de la aplicación

C En ambos sentidos, según convenga a cada caso de uso

D Desde la base de datos hacia los puertos de entrada

PREGUNTA

¿Cuál de estas afirmaciones sobre CQRS es correcta?

A Exige siempre dos bases de datos separadas

B Solo puede implementarse junto con Event Sourcing

C Puede aplicarse separando únicamente los modelos de lectura y escritura

D Sustituye a la arquitectura hexagonal en el núcleo

07

7. Errores, cierre y trabajo de la unidad

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

Los cinco errores que más cuestan al sustentar

1. Elegir microservicios por defecto
2. Confundir patrón arquitectónico con patrón GoF
3. Presentar un solo diagrama, sin vistas
4. Justificar con moda en vez de con RNF
5. Omitir las alternativas descartadas

...y los tres que arruinan la entrega

6. Ignorar las restricciones del negocio

7. Abstracciones con fuga

8. No decir qué se migra primero

Presupuesto limitado, dos desarrolladores y una ventana de dos horas son parte del problema, no ruido.

Del concepto al taller: qué sección alimenta qué entregable

Entregable	Se apoya en
A · Diagnóstico AS-IS	Sección 2: modularidad, separación, abstracción
B · Monolito vs distribuida	Sección 5 y el monolito modular
C · Propuesta TO-BE	Sección 6 (estilos) + sección 4 (patrones GoF)
D · Stack tecnológico	Sección 1: cada elección contra un RNF
E · Modelado C4	Error 3 + el manual de la unidad
F · Plan de migración	Sección 5: extracción selectiva
G · Presupuesto	Sección 3: toda decisión tiene costo

Tres ideas para llevarte

El costo manda

Arquitectura es lo que cuesta caro cambiar

Los números deciden

Un RNF sin número no descarta nada

Escribe la decisión

Sin alternativas descartadas, no hubo decisión

CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

¿Preguntas?

Jesús Ariel González Bonilla · ariel5253@hotmail.com